

Überblick über das CI/CD-Setup

Zur Unterstützung des Entwicklungsprozesses wurde eine Continuous-Integration-Pipeline (CI) mithilfe von GitHub Actions eingerichtet. Die Pipeline wird automatisch bei jedem Push auf das Repository ausgeführt und dient dazu, die erfolgreiche Kompilierung des Projekts sicherzustellen, automatisierte Tests auszuführen sowie Softwaremetriken zu erheben.

Ablauf der Pipeline

Der Workflow besteht aus mehreren aufeinanderfolgenden Schritten:

1. Auschecken des Repositorys

Zu Beginn wird der aktuelle Stand des Repositorys mithilfe der GitHub-Action `actions/checkout` in die Build-Umgebung geladen.

1. Einrichtung der Entwicklungsumgebung

Anschließend wird mit `actions/setup-java` eine Java-21-Laufzeitumgebung der Temurin-Distribution eingerichtet. Dadurch ist sichergestellt, dass alle Builds unter einer einheitlichen Java-Version erfolgen.

1. Projekt-Build und Testausführung

Das Projekt wird anschließend mit Maven über den Befehl `mvn clean package` kompiliert. Während dieses Build-Prozesses werden automatisch sämtliche implementierten JUnit-5-Unit-Tests ausgeführt. Schlägt ein Test fehl, wird die Pipeline beendet und der Build als fehlgeschlagen markiert.

1. Ermittlung von Softwaremetriken

Nach einem erfolgreichen Build wird das Open-Source-Werkzeug **CK (CKJM Extended)** aus dessen GitHub-Repository heruntergeladen und lokal kompiliert. Anschließend analysiert das Tool den Quellcode der drei Projektmodule:

- Server
- Client
- RogueCore

Für jedes Modul werden objektorientierte Softwaremetriken berechnet und als CSV-Dateien ausgegeben.

1. Zusammenführung der Ergebnisse

Die erzeugten Metrikdateien der einzelnen Module werden anschließend zu einer gemeinsamen Datei (`class.csv`) zusammengeführt. Dadurch entsteht eine zentrale Übersicht aller Klassenmetriken des gesamten Projekts.

1. Bereitstellung der Ergebnisse

Abschließend wird die zusammengeführte CSV-Datei als Build-Artefakt gespeichert. Dadurch können die Metriken auch nach Abschluss der Pipeline heruntergeladen und für weitere Auswertungen oder Dokumentationen verwendet werden.

Eingesetzte Werkzeuge

Werkzeug	Aufgabe
GitHub Actions	Automatisierung der Build-Pipeline
Maven	Build-Management und Testausführung
JUnit 5	Durchführung automatisierter Unit-Tests
OpenJDK 21 (Temurin)	Java-Laufzeitumgebung
CK	Berechnung objektorientierter Softwaremetriken

Bewertung

Die eingerichtete Pipeline automatisiert die wesentlichen Schritte der Qualitätssicherung. Jeder Commit wird unmittelbar kompiliert und getestet, wodurch Integrationsfehler frühzeitig erkannt werden. Zusätzlich ermöglicht die automatische Erhebung von Softwaremetriken eine kontinuierliche Bewertung der Codequalität hinsichtlich Komplexität, Kopplung und Klassenstruktur.

Eine vollständige Continuous-Deployment-Pipeline (CD) wurde im Rahmen des Projekts nicht umgesetzt, da keine automatische Bereitstellung oder Veröffentlichung der Anwendung vorgesehen war. Das eingesetzte Setup stellt daher eine Continuous-Integration-Pipeline mit automatisierter Qualitätsanalyse dar.